

Universitatea Națională de Știință și Tehnologie POLITEHNICA București

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

*Aplicație de monitorizare complexă a unui pacient la
nivelul unei unități medicale*

MED-MONITOR

Conducător științific

Prof. Dr. Ing. Radu RĂDESCU

Studenți

*Ing. Elena SORA
Ing. Cristi MILOIU*

2026

CUPRINS

Listă figuri.....	3
Listă tabele.....	4
Lista acronime	5
Introducere	6
1. Tehnologii Utilizate	7
1.1. Go si Framework-ul Gin.....	7
1.2. React si TypeScript.....	7
1.3. GORM si PostgreSQL.....	9
1.4. Casbin si Controlul Accesului Bazat pe Roluri (RBAC)	9
1.5. Google OAuth 2.0 si JWT	10
1.6. Docker si Containerizare	11
1.7. Genezio Cloud	12
1.8. Glassmorphism UI.....	12
2 Descrierea Proiectului	14
2.1 Structura Proiectului	14
2.2 Arhitectura Sistemului	15
2.3 Modulul de Autentificare si Autorizare	17
2.4 Modulul de Programari (Appointments)	19
2.5 Sistemul Digital de Prescriptii (RX).....	20
2.6 Frontend: Design Glassmorphism si Structura React	21
2.7 Disponibilitatea Sistemului și Accesibilitate Live	22
2.8 Securitate si Bune Practici	23
3. Concluzie.....	24
Bibliografie	25

Listă figuri

Fig. 1 Principiul de funcționare a Virtual DOM [7]	8
Fig. 2 Schema Logică a aplicație	15
Fig. 3 Pagina de Login a aplicației	17
Fig. 4 Inserarea unei noi programări	19
Fig. 5 Verificarea programării	19
Fig. 6 Vizualizarea prescripțiilor digitale din perspectiva pacientului	20
Fig. 7 Dashboard-ul aplicației	22

Listă tabele

Tabel 1 Descrierea structurii proiectului	14
Tabel 2 Roluri și Permisuni ale utilizatorilor	18

Lista acronime

ACID - Atomicity, Consistency, Isolation, Durability
API - Application Programming Interface
CORS - Cross-Origin Resource Sharing
CRUD - Create, Read, Update, Delete
CSS - Cascading Style Sheets
EMR - Electronic Medical Record
GDPR - General Data Protection Regulation
GORM - Go Object-Relational Mapper
HL7 FHIR - Health Level Seven - Fast Healthcare Interoperability Resources
HTTP / HTTPS - HyperText Transfer Protocol / Secure
I/O - Input/Output
JSON - JavaScript Object Notation
JWT - JSON Web Token
OAuth 2.0 - Open Authorization 2.0
ORM - Object-Relational Mapping
RBAC - Role-Based Access Control
SPA - Single Page Application
SQL - Structured Query Language
UI / UX - User Interface / User Experience
URL - Uniform Resource Locator

Introducere

Med-Monitor reprezintă un sistem enterprise de monitorizare medicală, conceput să fluidizeze întreaga activitate clinică dintr-un spital sau o clinică privată. Proiectul acordă o atenție deosebită performanței, securității și experienței utilizatorului, integrând cele mai recente practici de inginerie software pentru a transforma modul în care interacționează personalul medical și pacienții.

Sistemul de sănătate modern se confruntă cu o serie de provocări complexe legate de gestionarea eficientă a consultărilor, a prescripțiilor medicale și a dosarelor electronice ale pacienților. Lipsa unor instrumente digitale integrate generează adesea întârzieri birocratice, erori de comunicare între departamente și dificultăți majore în urmărirea traseului pacientului în cadrul unității medicale. Într-un mediu în care timpul este critic, fragmentarea informațiilor devine un obstacol în calea unui act medical de calitate.

Aplicația răspunde acestor provocări printr-o platformă web completă și intuitivă, structurată pe trei roluri distincte - Administrator, Medic și Pacient - fiecare beneficiind de un tablou de bord personalizat care maximizează productivitatea și relevanța datelor afișate. Autentificarea se realizează prin Google OAuth 2.0, asigurând un flux de înrolare fără fricțiuni și eliminând necesitatea gestionării manuale a parolelor complexe de către utilizatori. Securitatea este pilonul central al arhitecturii, unde controlul accesului bazat pe roluri (RBAC), implementat prin biblioteca Casbin, garantează că fiecare utilizator are acces exclusiv la datele și operațiunile permise, respectând cele mai riguroase standarde de confidențialitate.

Obiectivele principale ale proiectului vizează, în primul rând, digitalizarea integrală a ciclului clinic, acoperind întreg fluxul operațional, de la momentul rezervării inițiale până la eliberarea finală a prescripției medicale. Această transformare este susținută de o securitate robustă, bazată pe tehnologia JWT stateless combinată cu Google ID Token, oferind o protecție impenetrabilă a datelor sensibile. Dincolo de funcționalitatea pură, Med-Monitor redefineste interfața medicală tradițională printr-o experiență modernă de utilizare, adoptând un design vizual de tip Glassmorphism realizat integral în CSS pur, care oferă o estetică aerisită și profesională. Toate aceste componente sunt susținute de o arhitectură scalabilă și ușor de întreținut, containerizată prin Docker și implementată pe infrastructura Genezio Cloud, asigurând o disponibilitate ridicată și o performanță constantă indiferent de volumul de date procesat.

1. Tehnologii Utilizate

Această secțiune detaliază fundamentele teoretice și raționamentul tehnic din spatele principalelor tehnologii care susțin ecosistemul Med-Monitor, oferind contextul necesar pentru înțelegerea deciziilor arhitecturale care asigură scalabilitatea și reziliența sistemului.

1.1. Go si Framework-ul Gin

Limbajul Go (sau Golang) reprezintă alegerea strategică pentru nucleul sistemului Med-Monitor. Creat de Google pentru a răspunde provocărilor sistemelor distribuite de mari dimensiuni, Go este un limbaj compilat, cu tipizare statică, ce reușește să echilibreze performanța brută a limbajelor de nivel scăzut cu simplitatea și siguranța oferite de limbajele moderne [1]. În contextul unei aplicații medicale de tip enterprise, Go oferă avantaje critice prin sistemul său de gestionare a memoriei (Garbage Collection) extrem de eficient, care minimizează latențele în procesarea datelor în timp real.

Elementul definitoriu al limbajului Go este modelul de concurență bazat pe Goroutine-uri. Acestea sunt fire de execuție extrem de ușoare, gestionate de runtime-ul Go și nu direct de sistemul de operare, permițând serverului să gestioneze zeci de mii de conexiuni simultane - cum ar fi fluxurile de date ale pacienților sau interogările multiple ale medicilor - cu un consum de resurse hardware neglijabil [2].

Gin Web Framework completează puterea limbajului Go, servind drept motor de rutare ultra-rapid pentru Med-Monitor. Construit peste httprouter, Gin este recunoscut în industria software pentru amprenta sa minimă de memorie și viteza de execuție superioară, fiind optimizat pentru cerințele moderne de tip microservicii sau API-uri monolitice performante [3]. În arhitectura proiectului, Gin joacă rolul de orchestrator al stratului de transport, facilitând construirea unei interfețe API RESTful robuste.

Sistemul de Middleware permite interceptarea fiecărei cereri HTTP pentru operațiuni esențiale precum logarea activității, recuperarea automată din erori de tip panic și, cel mai important, validarea jetonului de acces înainte ca cererea să ajungă la logica de business. Prin mecanismele de Binding și Validare Declarativă, Gin automatizează procesul de transformare a datelor JSON primite de la utilizatori în structuri de date Go, aplicând reguli stricte de validare la nivel de câmp. Acest lucru este vital pentru a preveni introducerea de date eronate în dosarele medicale [4].

De asemenea, Gruparea Rutelor permite organizarea logică a API-ului pe grupuri de acces (ex: /api/v1/admin, /api/v1/medic), ceea ce facilitează aplicarea politicilor de securitate specifice fiecărui rol și simplifică evoluția ulterioară a sistemului. Datorită performanței sale predictibile, Gin garantează timpi de răspuns constanți, un factor esențial într-o aplicație clinică unde accesul instantaneu la informație este o necesitate critică [5].

1.2. React si TypeScript

Alegerea tehnologiilor pentru stratul de prezentare în aplicațiile software contemporane este ghidată de necesitatea de a construi interfețe care să fie nu doar estetice, ci și extrem de rezistente la erori, în special în domenii sensibile precum cel medical. În centrul acestei arhitecturi se află **React**, o bibliotecă declarativă bazată pe componente, care a schimbat fundamental modul în care sunt gândite aplicațiile web prin introducerea paradigmei de programare reactivă [6].

Conceptul fundamental care oferă React un avantaj competitiv este **Virtual DOM**. În loc să interacționeze direct cu structura documentului din browser la fiecare schimbare de stare - proces care consumă resurse hardware considerabile - React menține o copie virtuală a acesteia în memorie.

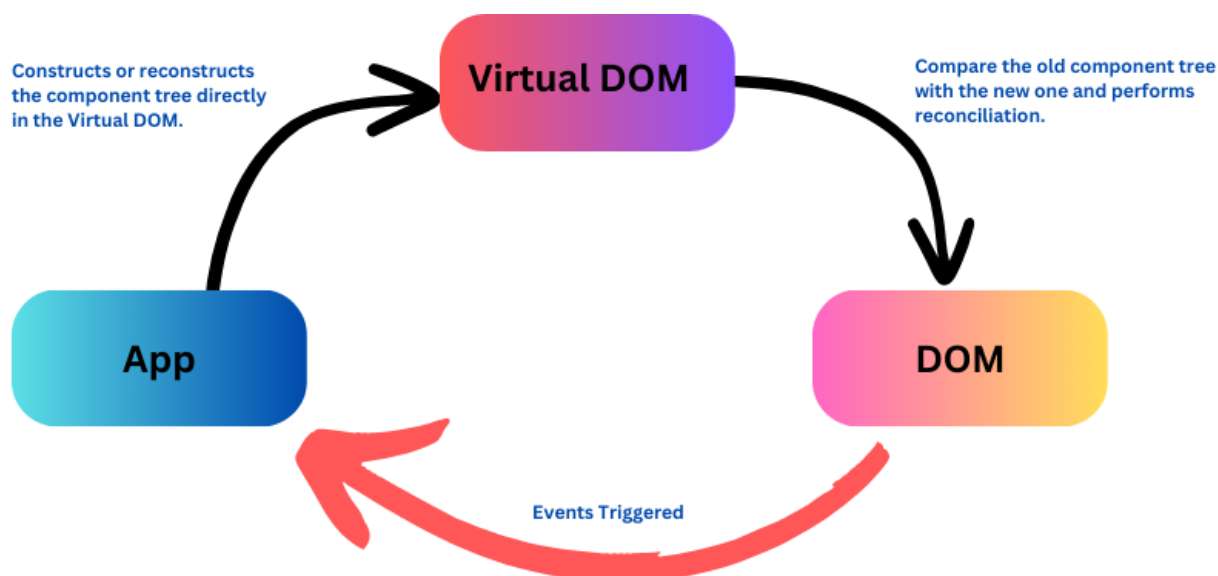


Fig. 1 Principiul de funcționare a Virtual DOM [7]

Prin procesul de *reconciliation*, biblioteca identifică doar diferențele strict necesare între starea anterioară și cea actuală, aplicând actualizările în mod chirurgical. Această abordare garantează o viteză de răspuns superioară, un factor critic în platformele care gestionează fluxuri mari de date dinamice în timp real, unde orice latență poate afecta productivitatea [7]. React 19 duce acest concept mai departe, optimizând gestionarea resurselor și randarea pe partea de client prin noi mecanisme automate de manipulare a operațiunilor asincrone.

Pentru a aduce rigoare inginerescă într-un mediu dominat istoric de natura dinamică a JavaScript-ului, integrarea **TypeScript 5** a devenit o necesitate pentru sistemele de tip enterprise. TypeScript reprezintă un supraset de tipizare statică ce transformă procesul de scriere a codului într-o experiență mult mai sigură și predictibilă [8]. Prin definirea strictă a interfețelor și a tipurilor de date, acesta permite identificarea erorilor logice încă din faza de scriere, mult înainte ca software-ul să ajungă în execuție. În contextul managementului de date critice, acest sistem de tipizare funcționează ca o barieră de protecție, asigurându-se că obiectele ce reprezintă informații complexe respectă structura impusă. Versiunea 5 a limbajului aduce îmbunătățiri majore în viteza de compilare și o granularitate mai mare în definirea tipurilor, facilitând o bază de cod modulară și ușor de refactorizat [9].

Eficiența procesului de dezvoltare și performanța finală a resurselor livrate în browser sunt asigurate de **Vite**, un instrument de build de ultimă generație. Vite adoptă o abordare radical diferită de predecesorii săi prin utilizarea **ESModules native** direct în browser în timpul etapei de dezvoltare [10]. Aceasta înseamnă că timpul de pornire al mediului de lucru rămâne constant, indiferent de dimensiunea proiectului, iar mecanismul de **Hot Module Replacement (HMR)** actualizează modulele modificate aproape instantaneu.

Prin fuziunea acestor trei piloni tehnologici, stratul de frontend este structurat ca un **Single Page Application (SPA)** modern, unde logica de navigare și randarea sunt gestionate eficient pe partea de client. Această arhitectură reduce sarcina asupra serverului și oferă utilizatorului o experiență fluidă, fără reîncărcări de pagină, menținând în același timp un nivel ridicat de securitate a codului prin contractele stricte impuse de tipizarea statică.

1.3. GORM si PostgreSQL

Asigurarea unei baze solide pentru stocarea și manipularea informațiilor reprezintă o componentă critică în arhitectura oricărui sistem software modern, cu atât mai mult în domenii unde integritatea, disponibilitatea și securitatea datelor sunt fundamentale. Pentru a gestiona aceste cerințe, se utilizează o combinație între un sistem de gestiune a bazelor de date relaționale extrem de robust și un strat de abstractizare care să faciliteze interacțiunea programatică eficientă.

PostgreSQL reprezintă standardul actual în materie de sisteme de gestiune a bazelor de date relaționale (RDBMS) open-source, fiind recunoscut pentru conformitatea sa riguroasă cu principiile **ACID** (Atomicitate, Consistență, Izolare, Durabilitate). Această conformitate garantează că toate tranzacțiile sunt procesate în siguranță, prevenind coruperea datelor chiar și în cazul unor eșecuri de sistem sau căderi de tensiune, un aspect vital în managementul informațiilor sensibile [11]. Dincolo de modelul relațional tradițional, PostgreSQL oferă suport nativ pentru tipuri de date complexe, precum **JSONB**, permițând o flexibilitate hibridă între structurile tabelare rigide și datele semi-structurate. În plus, capacitățile sale avansate de indexare și suportul pentru extensii precum **UUID-oss** (pentru identificatori unici universali) și tipuri de date de tip **ARRAY** îl transformă într-o soluție optimă pentru aplicații scalabile care necesită o precizie absolută a informației [11].

Pentru a interfața logica aplicației cu motorul PostgreSQL, se utilizează **GORM**, cel mai popular ecosistem de tip **Object-Relational Mapper (ORM)** pentru limbajul Go. GORM acționează ca un strat de mediere între structurile de date native ale limbajului și tabelele bazei de date, permițând dezvoltatorilor să manipuleze înregistrările folosind paradigme de programare orientată pe obiecte în locul scrierii manuale a interogărilor SQL complexe [12]. Această abstractizare nu doar că sporește productivitatea, dar reduce semnificativ riscul apariției vulnerabilităților de tip **SQL Injection**, deoarece biblioteca utilizează automat interogări parametrizate.

Principalele avantaje teoretice și practice ale utilizării GORM includ:

- **Migrarea Automată a Schemei:** GORM poate detecta automat modificările aduse structurilor de date în cod și poate actualiza schema bazei de date (tabele, coloane, constrângeri) fără a fi necesară intervenția manuală, asigurând sincronizarea constantă între cod și baza de date [13].
- **Gestionarea Relațiilor Complexe:** Biblioteca oferă suport nativ pentru definirea și interogarea relațiilor de tip *HasOne*, *HasMany*, *BelongsTo* și *Many2Many*. Prin mecanisme precum **Preloading** (Eager Loading), GORM optimizează performanța interogărilor, reducând numărul de cereri către baza de date prin utilizarea eficientă a operațiunilor de tip JOIN.
- **Hook-uri de Ciclu de Viață:** Dezvoltatorii pot defini funcții care să se execute automat înainte sau după anumite operațiuni (de exemplu, criptarea unei informații sensibile înainte de salvare sau logarea unui eveniment după ștergere), asigurând o trasabilitate completă a datelor.
- **Tranzacții și Consecvență:** GORM facilitează execuția mai multor operațiuni într-o singură unitate logică de lucru. Dacă oricare dintre pașii unei tranzacții eșuează, întregul proces este anulat, păstrând baza de date într-o stare consistentă [14].

Prin utilizarea PostgreSQL ca motor de persistență, sistemul beneficiază de un ecosistem capabil să gestioneze volume masive de date cu o fiabilitate de neegalat. În paralel, GORM oferă instrumentele necesare pentru o dezvoltare agilă, menținând un cod curat și o arhitectură ușor de întreținut, în care interacțiunea cu datele este previzibilă și sigură.

1.4. Casbin si Controlul Accesului Bazat pe Roluri (RBAC)

Securitatea datelor și granularitatea accesului reprezintă coloana vertebrală a oricărui sistem informatic complex, în special în domenii unde confidențialitatea informațiilor este reglementată prin standarde stricte. Pentru a asigura un management riguros al permisiunilor, se utilizează **Casbin**, o bibliotecă de autorizare open-source extrem de versatilă, care permite implementarea unor modele de control al accesului decuplate de logica de business a aplicației [15].

Spre deosebire de abordările tradiționale unde permisiunile sunt codificate rigid (hardcoded) în interiorul funcțiilor, Casbin utilizează un model bazat pe **PERM (Policy, Effect, Request, Matchers)**. Acest model permite definirea unei scheme de autorizare abstracte, unde accesul este determinat de

evaluarea unei cereri în raport cu o politică prestabilită. Un avantaj major al acestei abordări este flexibilitatea: politicile de acces pot fi stocate în fișiere externe (format .csv) sau în baze de date, permițând modificarea drepturilor de utilizator în timp real, fără a fi necesară recompilarea sau repornirea serviciilor [16].

Modelul **RBAC (Role-Based Access Control)**, implementat cu ajutorul Casbin, reprezintă o metodă de restricționare a accesului la resurse bazată pe rolurile atribuite utilizatorilor în cadrul unei organizații. În acest sistem, permisiunile nu sunt atribuite direct indivizilor, ci sunt asociate unor roluri specifice. Utilizatorii moștenesc apoi aceste permisiuni prin apartenența la unul sau mai multe roluri, simplificând considerabil administrarea securității în sisteme cu un număr mare de participanți [17].

În contextul unei arhitecturi software moderne, implementarea RBAC prin Casbin oferă beneficii fundamentale:

- **Ierarhia Rolurilor:** Casbin permite definirea unor structuri ierarhice (de exemplu, un rol de super-administrator care moștenește toate permisiunile rolurilor subordonate), reducând redundanța în definirea politicilor [18].
- **Integrarea ca Middleware:** Prin integrarea cu framework-uri web, procesul de autorizare are loc la nivelul stratului de transport. Fiecare cerere HTTP este interceptată și validată înainte de a ajunge la datele sensibile, asigurând o protecție proactivă a rutelor API.
- **Separarea Responsabilităților:** Dezvoltatorii se pot concentra pe funcționalitate, în timp ce administratorii de sistem pot gestiona politicile de securitate independent, prin simpla actualizare a fișierului de politici sau a tabelelor dedicate în baza de date.

În sistemele de gestionare a informațiilor, acest model asigură un flux de lucru coerent și protejat. Un administrator de sistem deține autoritate deplină asupra configurărilor și managementului utilizatorilor, în timp ce personalul specializat (cum este cazul medicilor) primește acces exclusiv la instrumentele de diagnostic, prescripție și vizualizare a istoricului clinic. La celălalt capăt al spectrului, utilizatorii finali sau pacienții beneficiază de un acces restricționat, limitat strict la interacțiunile proprii, precum programările sau vizualizarea propriilor documente medicale. Această barieră digitală nu doar că protejează împotriva accesului neautorizat, dar și minimizează riscul de eroare umană prin limitarea acțiunilor disponibile fiecărei categorii de utilizatori [19].

1.5. Google OAuth 2.0 si JWT

Securitatea identității și protecția accesului reprezintă elemente critice în dezvoltarea sistemelor informatice care manipulează date sensibile. Într-o arhitectură modernă, procesul de autentificare nu se mai limitează la simpla verificare a unei combinații de utilizator și parolă, ci adoptă standarde de industrie care maximizează siguranța și îmbunătățesc experiența utilizatorului final.

OAuth 2.0 reprezintă standardul de aur în materie de autorizare delegată, permițând unei aplicații să acceseze resurse în numele unui utilizator, fără ca acesta din urmă să își divulge acreditările (parola) către aplicația respectivă [20]. Prin utilizarea **Google OAuth 2.0**, procesul de înrolare devine nu doar mai rapid, ci și considerabil mai sigur. Utilizatorul interacționează direct cu infrastructura Google, care, în urma unei autentificări reușite, furnizează aplicației un **ID Token**. Acest jeton, semnat digital, conține informații verificate despre identitatea utilizatorului, eliminând riscurile asociate cu stocarea locală a parolelor și gestionarea vulnerabilităților aferente fluxurilor de autentificare proprii [21].

Odată ce identitatea utilizatorului a fost confirmată prin intermediul furnizorului de identitate (IdP), gestiunea sesiunii de lucru este preluată de standardul **JSON Web Token (JWT)**. Definit prin specificația **RFC 7519**, JWT este un format compact și autonom pentru transmiterea securizată a informațiilor între două părți sub forma unui obiect JSON [22]. Un jeton JWT este compus din trei secțiuni distincte: **Header** (specifică algoritmul de semnare), **Payload** (conține declarațiile sau „claims-urile” despre utilizator, cum ar fi ID-ul sau rolul acestuia) și **Signature** (garantează integritatea datelor și faptul că jetonul nu a fost alterat pe parcursul transmisiei).

Un avantaj arhitectural major al JWT este natura sa **stateless** (fără stare). Spre deosebire de sistemele tradiționale de sesiune care necesită stocarea unui identificator într-o bază de date sau într-un cache (precum Redis) pentru fiecare utilizator activ, un backend care utilizează JWT nu trebuie să mențină o evidență a sesiunilor pe server [23]. Toate informațiile necesare pentru autorizarea unei cereri sunt incluse

chiar în jetonul trimis de client. Acest aspect facilitează **scalarea orizontală** a sistemului: orice instanță a serverului din spatele unui load balancer poate valida în mod independent jetonul prin verificarea semnăturii digitale folosind o cheie secretă partajată sau o cheie publică.

În fluxurile de lucru care necesită un nivel înalt de disponibilitate, combinația dintre Google OAuth 2.0 și JWT oferă un echilibru optim:

- **Securitate sporită:** Identitatea este garantată de un furnizor global de încredere, reducând suprafața de atac.
- **Performanță:** Validarea jetonului JWT se face local, la nivelul serverului, fără a mai efectua interogări suplimentare către baze de date pentru fiecare cerere HTTP.
- **Interoperabilitate:** Standardele deschise permit integrarea facilă cu diverse platforme și servicii terțe, asigurând o arhitectură pregătită pentru evoluții viitoare [24].

1.6. Docker si Containerizare

În ingineria software modernă, discrepanțele dintre mediile de dezvoltare, testare și producție reprezintă una dintre cele mai frecvente surse de erori tehnice. Pentru a elimina această problemă, procesul de implementare se bazează pe tehnologia de containerizare, unde **Docker** s-a impus ca standard de facto în industrie. Docker permite împachetarea unei aplicații împreună cu toate bibliotecile, variabilele de mediu și fișierele de configurare necesare într-o unitate logică izolată numită container [25].

Spre deosebire de mașinile virtuale tradiționale, containerele Docker nu includ un sistem de operare complet, ci partajează nucleul (kernel-ul) sistemului gazdă. Această abordare face ca unitățile de execuție să fie extrem de ușoare, pornind în doar câteva secunde și consumând o fracțiune din resursele hardware necesare anterior. În ecosistemele bazate pe microservicii sau în arhitecturile monolitice moderne, containerizarea asigură faptul că software-ul va rula identic indiferent de infrastructura pe care este deployat, fie că este vorba de un laptop personal, un server on-premise sau o platformă complexă de tip cloud [26].

Gestionarea manuală a mai multor containere care trebuie să interacționeze între ele (cum ar fi un server de backend, o interfață de frontend și o bază de date) poate deveni rapid o sarcină redundantă și predispusă la greșeli. Pentru a rezolva această provocare, se utilizează **Docker Compose**, un instrument de orchestrare locală care permite definirea și rularea aplicațiilor multi-container. Printr-un singur fișier de configurare în format YAML, se poate descrie întreaga stivă tehnologică, specificând rețelele virtuale de comunicare între servicii, volumele de date pentru persistență și politicile de repornire în caz de eroare [27].

Un aspect fundamental în utilizarea acestor tehnologii este optimizarea fluxului de lucru prin utilizarea **volumelor montate** (Bind Mounts). Această tehnică permite maparea unui director din sistemul de fișiere al dezvoltatorului direct în interiorul containerului aflat în execuție. Astfel, orice modificare adusă codului sursă este reflectată instantaneu în mediul containerizat, facilitând mecanisme precum *Hot Module Replacement* sau *Live Reload* fără a fi necesară reconstrucția laborioasă a imaginilor Docker la fiecare pas [28].

Containerizarea nu contribuie doar la agilitatea dezvoltării, ci și la securitatea generală a sistemului. Fiecare container rulează într-un spațiu de procese izolat, limitând astfel impactul unei eventuale vulnerabilități la nivelul unui singur serviciu. Prin definirea unor limite clare de resurse (CPU, RAM) și izolarea rețelelor, infrastructura devine mult mai rezistentă și mai ușor de monitorizat, oferind o bază stabilă pentru aplicațiile care necesită o disponibilitate ridicată și o scalabilitate orizontală rapidă în funcție de cerințele de trafic [29].

1.7. Genezio Cloud

Tranziția de la un mediu de dezvoltare local la o infrastructură accesibilă global reprezintă o etapă critică în ciclul de viață al oricărei aplicații software. În prezent, tendința majoră în ingineria software este adoptarea soluțiilor de tip **Platform as a Service (PaaS)**, care permit dezvoltatorilor să se concentreze exclusiv pe scrierea codului, delegând gestionarea complexă a serverelor, a rețelelor și a securității infrastructurii către furnizorul de cloud. O astfel de soluție este **Genezio**, o platformă cloud inovatoare care optimizează procesul de deployment pentru aplicațiile full-stack moderne [30].

Genezio se distinge prin abordarea sa centrată pe productivitate, oferind un ecosistem care reduce semnificativ timpul necesar configurării manuale a mediilor de producție. Unul dintre avantajele sale fundamentale este integrarea nativă cu sistemele de versionare precum **GitHub**. Această conexiune facilitează implementarea fluxurilor de tip **Continuous Integration/Continuous Deployment (CI/CD)**, unde fiecare actualizare a codului sursă este preluată automat, testată și publicată în instanțele de producție fără intervenție umană. Pentru sistemele care operează în regiuni cu reglementări stricte privind datele, prezența infrastructurii în centre de date din Europa Centrală reprezintă un beneficiu major, asigurând atât conformitatea cu normele de protecție a datelor, cât și latențe minime pentru utilizatorii din această zonă geografică [31].

Din punct de vedere tehnic, platforma aduce inovații substanțiale în modul în care comunică straturile de frontend și backend:

- **Generarea Automată de SDK-uri:** Genezio analizează codul de backend (în cazul nostru, scris în Go) și generează automat biblioteci de client (SDK-uri) pentru frontend. Acest mecanism elimină necesitatea scrierii manuale a apelurilor API și asigură o comunicare de tip **type-safe**. Orice modificare în structura datelor de pe server este reflectată imediat în codul de client, prevenind erorile de sincronizare.
- **Scalabilitate Automată:** Infrastructura este proiectată să gestioneze variațiile de trafic în mod dinamic. Platforma poate suplimenta resursele alocate în momentele de vârf și le poate reduce în perioadele de inactivitate, optimizând astfel consumul de energie și costurile operaționale.
- **Securitatea Stratului de Transport:** Toate endpoint-urile sunt securizate automat prin certificate SSL/TLS, garantând că transferul de date între utilizatorul final și infrastructura cloud este criptat conform standardelor de securitate cibernetică [31].

Prin utilizarea acestor tehnologii de cloud, aplicația beneficiază de o disponibilitate ridicată și de un proces de actualizare transparent. Separarea clară a instanțelor - una dedicată livrării activelor statice de frontend și una specializată în execuția logicii de business din backend - permite o mentenanță granulară și o izolare a resurselor. Această arhitectură cloud-native asigură faptul că sistemul poate evolua rapid, integrând funcționalități noi într-un mediu stabil, securizat și performant, capabil să răspundă cerințelor exigente ale domeniului tehnologic actual [32].

1.8. Glassmorphism UI

Evoluția designului interfețelor digitale a migrat constant de la minimalismul plat către stiluri care încearcă să reproducă profunzimea și textura obiectelor reale într-un mod abstract. **Glassmorphism** reprezintă una dintre cele mai proeminente tendințe vizuale contemporane, fiind definită printr-un efect de „sticlă mată” care oferă interfețelor o senzație de stratificare, modernitate și eleganță. Acest stil nu este doar o alegere estetică, ci și una funcțională, ajutând la stabilirea unei ierarhii vizuale clare prin utilizarea transparenței și a profunzimii [33].

Din punct de vedere teoretic și tehnic, Glassmorphismul se bazează pe câteva proprietăți vizuale fundamentale:

- **Translucența și Efectul de Blur:** Caracteristica principală este fundalul semi-transparent care lasă să se întrevadă culorile din spate, dar le difuzează folosind proprietatea backdrop-filter: blur(). Acest efect creează o distincție clară între planul de lucru și fundal, menținând în același timp contextul vizual general [34].
- **Stratificarea Ierarhică:** Prin utilizarea unor umbre difuze și a unor borduri extrem de subtile, adesea albe și parțial transparente, se creează iluzia că elementele de interfață plutesc pe diferite

niveluri de profunzime (Z-axis). Acest lucru facilitează orientarea utilizatorului în cadrul aplicațiilor complexe [35].

- **Contrastul și Lizibilitatea:** Pentru a compensa fundalurile variabile, acest stil pune un accent deosebit pe tipografie și pe culori de accent vibrante, asigurându-se că textul rămâne lizibil indiferent de gradul de transparență al elementelor părinte.

Spre deosebire de fluxurile de lucru convenționale care se bazează pe biblioteci de componente pre-stilizate precum Material UI, Bootstrap sau Tailwind CSS, implementarea Glassmorphismului în format **CSS pur** (Vanilla CSS) oferă un control total asupra fiecărui pixel. Această abordare elimină dependențele externe masive, reducând dimensiunea pachetului final de resurse și eliminând conflictele de stilizare care pot apărea în bibliotecile de componente standardizate [36].

Utilizarea CSS-ului nativ pentru realizarea acestui efect permite exploatarea funcțiilor moderne ale browserelor, cum ar fi variabilele CSS (Custom Properties), care facilitează schimbarea temelor vizuale la nivel global cu un impact minim asupra performanței de randare. De asemenea, independența față de framework-urile de design asigură o identitate vizuală unică, „premium”, care nu urmează tiparele vizuale repetitive întâlnite în majoritatea aplicațiilor web actuale. Prin controlul direct asupra proprietăților de filtrare și a modelelor de culori RGBA, se obține o interfață fluidă, care se adaptează armonios la diverse rezoluții și dispozitive, păstrând în același timp o estetică profesională și aerisită, specifică soluțiilor software de înaltă calitate [37].

2 Descrierea Proiectului

Această secțiune oferă o analiză aprofundată a implementării tehnice, detaliind arhitectura sistemului, organizarea modulară a codului sursă și fluxurile funcționale care permit operarea platformei. În dezvoltarea sistemelor software complexe, modul în care este structurată baza de cod influențează direct mentenabilitatea, scalabilitatea și ușurința cu care noi funcționalități pot fi integrate fără a perturba procesele existente.

2.1 Structura Proiectului

Proiectul adoptă o strategie de organizare de tip **monorepo**, o arhitectură ce permite gestionarea ambelor componente fundamentale - backend-ul și frontend-ul - în cadrul aceluiași depozit de cod (repository). Această abordare simplifică procesele de Continuous Integration (CI), permite o vizibilitate sporită asupra modificărilor care afectează întregul stack și facilitează orchestrarea serviciilor prin instrumente de containerizare. Structura este concepută pentru a respecta principiul **Separării Responsabilităților** (Separation of Concerns), asigurând un decuplaj clar între logica de business, stratul de prezentare și mecanismele de persistență a datelor.

Tabel 1 Descrierea structurii proiectului

Director / Fisier	Descriere
backend/	Reprezintă nucleul logic al sistemului, găzduind un API RESTful robust scris în limbajul Go. Acesta integrează framework-ul Gin pentru rutare, GORM pentru comunicarea cu baza de date și Casbin pentru gestionarea politicilor de securitate.
backend/handlers/	Constituie stratul de interfață al serverului. Aici sunt definite punctele de acces (endpoints) HTTP, responsabilitatea acestora fiind preluarea cererilor, validarea preliminară a parametrilor de intrare și returnarea răspunsurilor către client în format JSON.
backend/services/	Găzduiește logica de business pură, independentă de protocoalele de transport. Separarea serviciilor de handleri asigură faptul că regulile de procesare a datelor medicale pot fi testate unitar și reutilizate fără a depinde de contextul unei cereri HTTP.
backend/repositories/	Implementează modelul <i>Repository Pattern</i> , oferind un strat de abstractizare peste baza de date. Toate interogările complexe și operațiunile de scriere prin GORM sunt centralizate aici, protejând restul aplicației de detaliile specifice ale schemei SQL.
backend/casbin/	Conține configurația motorului de autorizare. Fișierul model.conf definește structura cererilor de acces, în timp ce policy.csv reprezintă baza de date de permisiuni, unde sunt mapate rolurile utilizatorilor la resursele și acțiunile permise.
backend/Dockerfile	Fișierul de configurare care descrie procesul de construcție a imaginii Docker pentru mediul Go, asigurând un mediu de rulare izolat, optimizat pentru performanță și securitate.
frontend/	Găzduiește aplicația de tip Single Page Application (SPA), construită cu React 19 și TypeScript. Această

	componentă este responsabilă pentru randarea interfeței Glassmorphism și gestionarea stării de utilizare pe partea de client.
frontend/src/	Inima interfeței utilizator, unde sunt organizate componentele reutilizabile, paginile principale, hook-urile personalizate pentru logica React și interceptorii Axios utilizați pentru comunicarea securizată cu API-ul.
frontend/Dockerfile	Definește procesul de build pentru mediul de frontend, bazat pe Node.js, transformând codul TypeScript în active statice (static assets) optimizate pentru a fi servite prin intermediul unui server web de înaltă performanță.
docker-compose.yml	Fișierul de orchestrare care definește modul în care backend-ul și frontend-ul sunt legate într-o rețea virtuală comună, gestionând variabilele de mediu și porturile de comunicare pentru o rulare locală identică cu cea din producție.
docs/	Un spațiu dedicat resurselor de suport, incluzând diagrame de arhitectură, scheme ale bazei de date și capturi de ecran care documentează evoluția vizuală a platformei.
README.md	Documentul principal de referință, care oferă o prezentare generală a tehnologiilor utilizate, cerințele de sistem și pașii necesari pentru instalarea, securizarea și lansarea proiectului de către alți dezvoltatori.

Această structură ierarhică permite o navigare intuitivă în baza de cod și asigură o scalabilitate ridicată, permițând echipelor de dezvoltare să lucreze simultan pe componente diferite fără a genera conflicte structurale majore. Prin separarea clară a accesului la date (repositories), a logicii de prelucrare (services) și a interfeței cu utilizatorul (handlers/frontend), sistemul devine nu doar mai ușor de înțeles, ci și mult mai rezistent la schimbări tehnologice viitoare.

2.2 Arhitectura Sistemului

Sistemul **Med-Monitor** este fundamentat pe o arhitectură client-server modernă, organizată pe două niveluri logice principale care asigură o decuplare totală între interfața utilizator și logica de procesare a datelor. Această structură este proiectată să fie „stateless”, eliminând dependența de sesiuni stocate pe server și permițând astfel o scalare orizontală facilă, precum și o securitate riguroasă la fiecare punct de tranzit al informației medicale.

Reprezentarea Vizuală a Fluxurilor de Date

Diagrama de mai jos ilustrează parcursul bidirecțional al unei cereri, evidențiind separarea responsabilităților între stratul de acces, mecanismele de securitate și nucleul operațional de procesare:

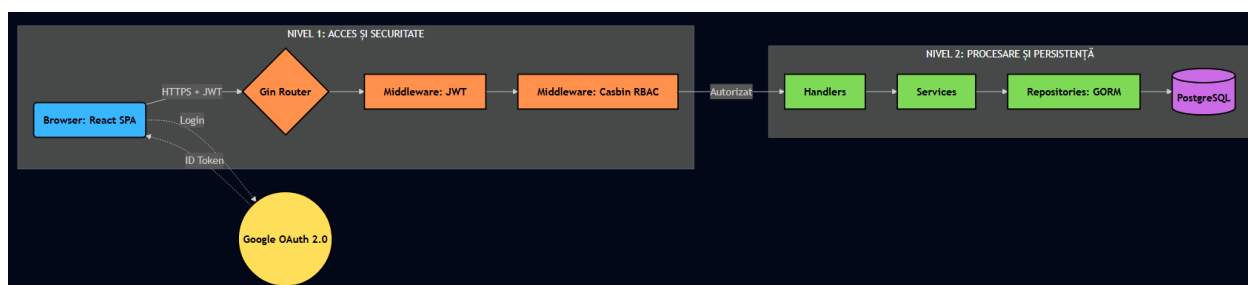


Fig. 2 Schema Logică a aplicației

Analiza Detaliată a Fluxului Funcțional

Arhitectura este structurată ierarhic pentru a asigura integritatea datelor și validarea drepturilor de acces înainte ca orice operațiune să interacționeze cu stratul de persistență. Această abordare minimizează suprafața de atac și garantează consistența informațiilor clinice.

1. Nivelul de Acces și Securitate (Poarta de Intrare) Acest prim nivel reprezintă stratul de interfață și filtrare. Orice cerere inițiată din frontend-ul **React** este tratată ca fiind potențial nesigură până la trecerea prin filtrele de identitate:

- **Identitate Delegată (Google OAuth 2.0):** Prin utilizarea unui furnizor de identitate extern, sistemul evită riscurile asociate stocării parolelor. Procesul de *handshake* garantează că doar utilizatorii cu conturi valide pot genera un ID Token valid.
- **Routerul Gin (Orchestrare):** Funcționează ca un dispecer central care primește traficul HTTPS și îl direcționează către lanțul de middleware-uri. Acesta asigură că cererile malformate sunt respinse înainte de a consuma resurse de procesare.
- **Validarea JWT (Stateless Auth):** Middleware-ul de securitate decriptează și verifică semnătura jetonului JWT. Deoarece jetonul conține toate informațiile necesare identificării (ID utilizator, rol), serverul nu trebuie să interogheze baza de date pentru a confirma sesiunea, optimizând drastic timpul de răspuns.
- **Controlul Casbin RBAC (Autorizare Granulară):** Aceasta este componenta care aplică politicile clinice. Chiar dacă un utilizator este autentificat, Casbin verifică dacă rolul său îi permite acțiunea solicitată (ex: un Pacient nu poate accesa endpoint-ul de modificare a prescripțiilor altui utilizator).

2. Nivelul de Procesare și Persistență (Nucleul Operațional) După ce validările de securitate sunt completate, cererea intră în zona logicii de aplicație, unde datele brute sunt transformate în informație medicală utilă:

- **Handlers (Gestionarea I/O):** Acest strat se ocupă de „traducerea” cererii HTTP în structuri de date native Go. Handler-urile extrag parametrii din URL sau corpul JSON și se asigură că datele de intrare respectă formatul tehnic așteptat, oferind un prim nivel de validare a datelor (Data Sanitization).
- **Services (Logica de Business și Reguli Clinice):** Reprezintă segmentul cel mai important al backend-ului. Aici sunt implementate regulile care definesc activitatea medicală: algoritmi de verificare a disponibilității medicilor, fluxuri de aprobare a rețetelor și procesarea dosarelor electronice. Serviciile sunt complet izolate de protocoalele de rețea, facilitând testarea unitară riguroasă.
- **Repositories (Abstractizarea Datelor):** Utilizând **GORM**, acest strat transformă apelurile de funcții Go în interogări SQL optimizate. Repositories asigură faptul că logica de business nu depinde de structura tabelor din baza de date, oferind o interfață curată pentru operațiunile de tip CRUD.
- **PostgreSQL (Persistența ACID):** Punctul final al fluxului unde datele sunt stocate permanent. Fiind un sistem tranzacțional, acesta garantează că o operațiune complexă (de exemplu, salvarea unei consultații și actualizarea istoricului simultan) ori se execută complet, ori deloc, eliminând riscul de date parțiale sau corupte.

Această separare funcțională pe două niveluri permite o mentenanță granulară: echipa de frontend poate evolua independent, în timp ce logica de securitate și regulile de business rămân protejate și neschimbate în backend, asigurând un sistem rezilient, ușor de auditat și pregătit pentru cerințele viitoare ale mediului medical digitalizat.

2.3 Modulul de Autentificare si Autorizare

Securitatea sistemului este construită pe doi piloni fundamentali: **autentificarea**, care garantează identitatea utilizatorului, și **autorizarea**, care controlează riguros acțiunile pe care acesta le poate efectua. Implementarea adoptă un model de tip *zero-friction*, eliminând barierele clasice (precum crearea și memorarea unor parole noi) și delegând responsabilitatea securității către infrastructura Google.

Fluxul de Autentificare (Identity Management)

Procesul de acces este conceput să fie fluid și sigur, urmând standardele moderne de comunicare între client, server și furnizorul de identitate:

1. **Inițierea (Single Sign-On):** Utilizatorul interacționează cu interfața prin butonul „Sign in with Google”. Navigatorul este redirecționat către ecranul de consimțământ Google (*Google Consent Screen*), asigurând utilizatorul că datele sale de acces rămân private și nu sunt partajate cu aplicația.
2. **Validarea și Schimbul de Token-uri:** După aprobare, Google furnizează un **ID Token**. Frontend-ul transmite acest token către endpoint-ul dedicat `/auth/google`. Backend-ul validează jetonul utilizând bibliotecile oficiale Google OAuth2, extrăgând datele de profil (email, nume, imagine).
3. **Generarea Sesiunii (JWT):** Odată identitatea confirmată, serverul emite un **JSON Web Token (JWT)** propriu, semnat digital cu o cheie secretă. Acest jeton devine „pașaportul” utilizatorului în interiorul ecosistemului, conținând informații despre rolul și identitatea acestuia.

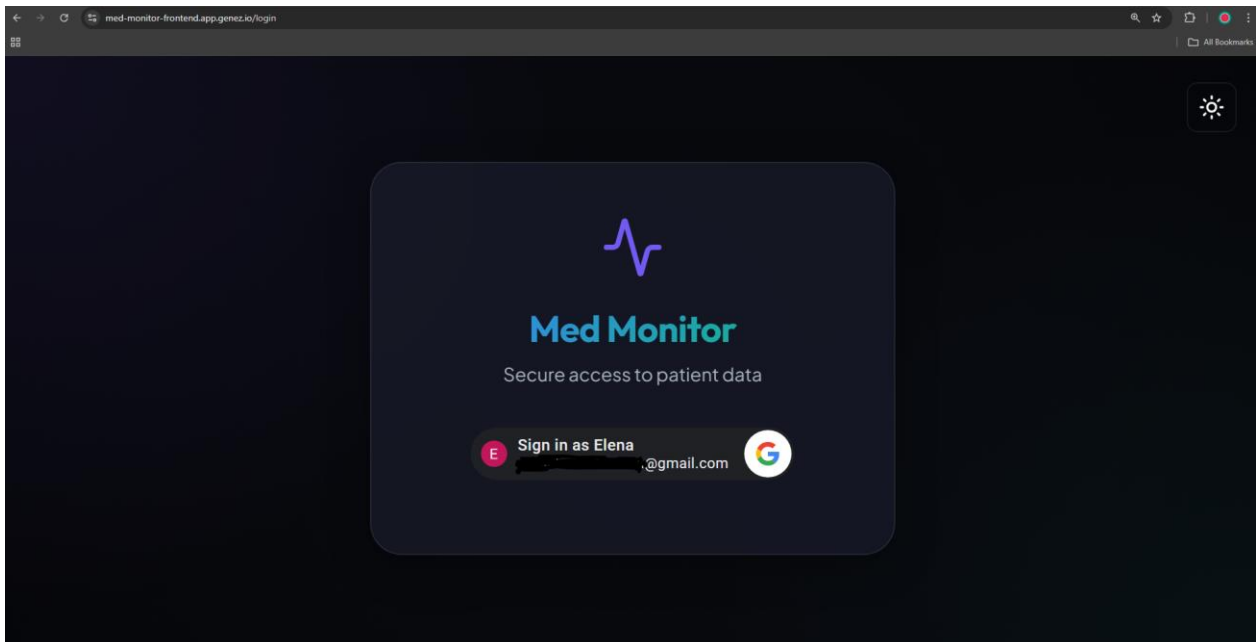


Fig. 3 Pagina de Login a aplicației

Mecanismul de Autorizare (Access Control)

După autentificare, fiecare interacțiune este supravegheată de un strat de securitate dublu, integrat ca middleware în framework-ul Gin:

- **Validarea Jetonului:** Fiecare cerere HTTP ulterioară trebuie să includă JWT-ul în header-ul de tip `Authorization: Bearer <token>`. Serverul verifică integritatea și valabilitatea acestuia înainte de a permite accesul la orice logică de business.
- **Evaluarea Politicilor (Casbin):** Motorul Casbin analizează cererea în timp real, comparând rolul utilizatorului, resursa solicitată (URL) și metoda HTTP (GET, POST, etc.) cu matricea de permisiuni predefinită în `policy.csv`.

Roluri și Permișiuni

Sistemul implementează un model **RBAC (Role-Based Access Control)** ierarhic, adaptat nevoilor specifice ale unui ecosistem de monitorizare medicală:

Tabel 2 Roluri și Permișiuni ale utilizatorilor

Rol	Permișiuni cheie
Admin	Detine control total asupra sistemului. Are capacitatea de a crea sau șterge conturi de utilizatori, de a gestiona structura departamentelor clinice și de a audita activitatea globală a platformei.
Doctor	Rol operațional medical. Permișiunile sunt centrate pe actul medical: înregistrarea fișelor de consultație, eliberarea prescripțiilor electronice și accesul la dosarul medical electronic (EMR) al pacienților din evidență.
Patient	Rol de utilizator final. Accesul este strict limitat la datele personale: rezervarea și gestionarea programărilor, vizualizarea istoricului propriilor prescripții și actualizarea profilului personal.

Prin utilizarea acestui model de control al accesului bazat pe roluri (**RBAC**), sistemul garantează o separare strictă a responsabilităților, asigurând totodată că datele medicale sensibile sunt accesate doar în conformitate cu necesitățile profesionale sau personale ale fiecărei categorii de utilizatori.

Implementarea ierarhică și securitatea datelor

Modelul ierarhic prezentat în **Tabelul 1** nu servește doar la organizarea interfeței, ci constituie fundamentul securității la nivel de bază de date. De exemplu, un **Admin** deține privilegii la nivel de infrastructură (gestiunea utilizatorilor și a departamentelor), însă politicile sistemului pot fi configurate astfel încât acesta să nu poată accesa conținutul clinic privat al consultațiilor, respectând principiul separării datelor administrative de cele medicale.

Personalizarea experienței utilizatorului

Din punct de vedere funcțional, autorizarea influențează direct fluxul de lucru în aplicație:

- **Pentru Medici:** Sistemul devine un instrument de lucru activ, oferind acces rapid la istoricul pacientului și instrumente de scriere (prescripții, note).
- **Pentru Pacienți:** Interfața este simplificată, transformându-se într-un portal de vizualizare și autogestionare, unde singurele acțiuni de scriere permise sunt cele legate de propriile programări.

Această granularitate, gestionată prin motorul **Casbin**, permite extinderea facilă a sistemului. Dacă în viitor va fi necesară adăugarea unui nou rol (de exemplu, „Asistent Medical”), acesta va putea fi integrat prin simpla definire a unui nou set de reguli în politicile de acces, fără a rescrie logica de funcționare a backend-ului.

2.4 Modulul de Programari (Appointments)

Gestiunea fluxului de pacienți reprezintă o componentă vitală a sistemului **Med-Monitor**, acest modul fiind responsabil pentru orchestrarea interacțiunilor dintre medici și pacienți. Implementarea tehnică pune accent pe sincronizarea datelor în timp real și pe o interfață intuitivă, menită să reducă timpul de așteptare și să optimizeze programul de lucru al personalului medical.

Fluxul de Rezervare pentru Pacienți

Modulul este construit în jurul unui **calendar interactiv**, care acționează ca un punct central de coordonare. Pacienții beneficiază de o experiență transparentă, putând vizualiza în timp real sloturile de disponibilitate ale fiecărui medic specialist. Procesul de rezervare este simplificat: utilizatorul selectează medicul dorit, alege un interval orar marcat ca fiind liber și confirmă programarea. Această vizibilitate elimină erorile de tip "double booking" și oferă utilizatorului autonomia de a-și gestiona vizitele medicale fără intermediari.

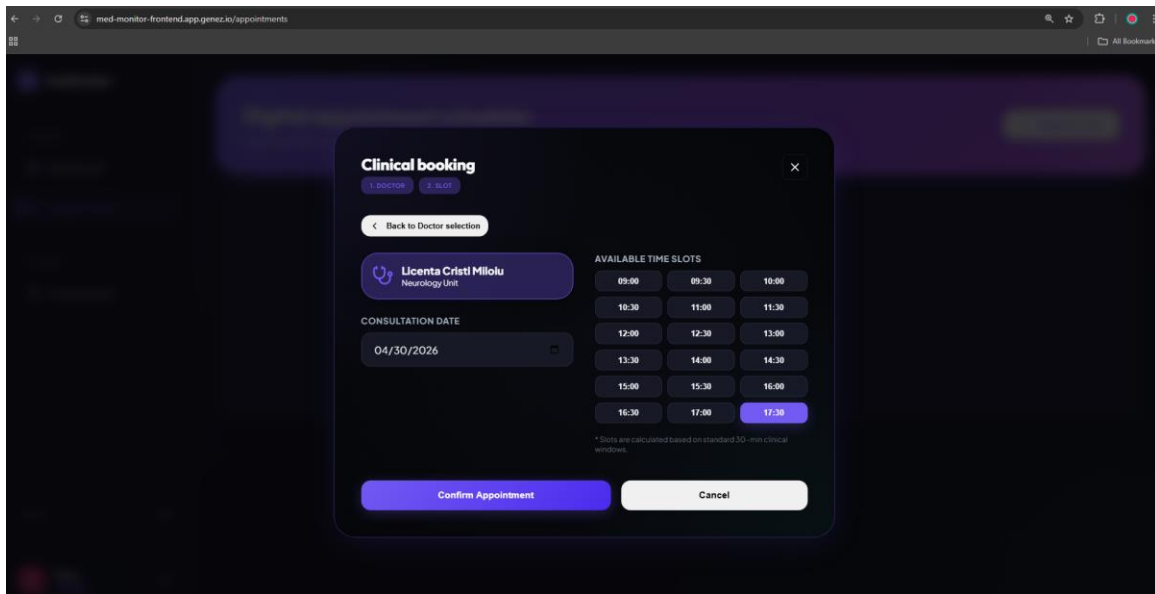


Fig. 4 Inserarea unei noi programări

Mai apoi, se poate verifica programarea în secțiunea de Appointments

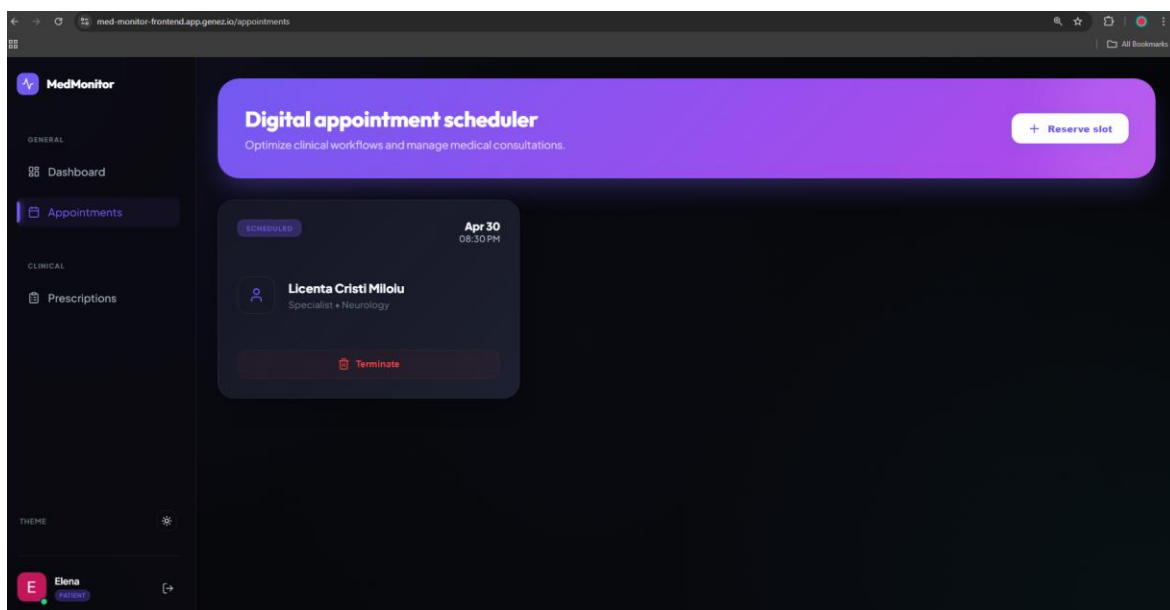


Fig. 5 Verificarea programării

2.5 Sistemul Digital de Prescriptii (RX)

Digitalizarea procesului de eliberare a rețetelor medicale reprezintă una dintre funcționalitățile nucleu ale platformei **Med-Monitor**, având ca scop eliminarea erorilor de scriere manuală și facilitarea accesului instantaneu al pacientului la tratamentul recomandat. Acest modul transformă experiența tradițională a „rețetei pe hârtie” într-un flux de date securizat, centralizat și ușor de gestionat.

Fluxul Operațional și Validarea Datelor

Procesul începe în dashboard-ul medicului, unde acesta poate emite o prescripție nouă printr-o interfață dedicată. Medicul selectează medicamentul, doza specifică și durata tratamentului, datele fiind trimise către backend-ul Go. Aici, sistemul efectuează o serie de validări riguroase înainte de a salva informația în baza de date **PostgreSQL**, asigurându-se că toate câmpurile sunt completate corect și că prescripția este legată de un profil de pacient valid.

Odată stocată, prescripția devine o parte permanentă a istoricului medical, fiind disponibilă instantaneu în contul utilizatorului vizat.

Experiența Pacientului: Accesibilitate și Format Ready-to-Print

Din perspectiva **Pacientului**, modulul este optimizat pentru simplitate și utilitate practică. În secțiunea dedicată din aplicație, pacientul poate regăsi lista tuturor prescripțiilor emise pe numele său, organizate cronologic.

Interfața de vizualizare folosește stilul **Glassmorphism** pentru a menține estetica premium a platformei, dar pune accent pe claritatea informației clinice. Pacientul are posibilitatea de a deschide detaliile fiecărei prescripții într-un format **"Ready-to-Print"**. Această funcționalitate asigură că informațiile esențiale (numele medicului, unitatea medicală, medicația și instrucțiunile) sunt prezentate clar, permițând pacientului să prezinte rețeta la farmacie direct de pe dispozitivul mobil sau să o imprime dacă este necesar.

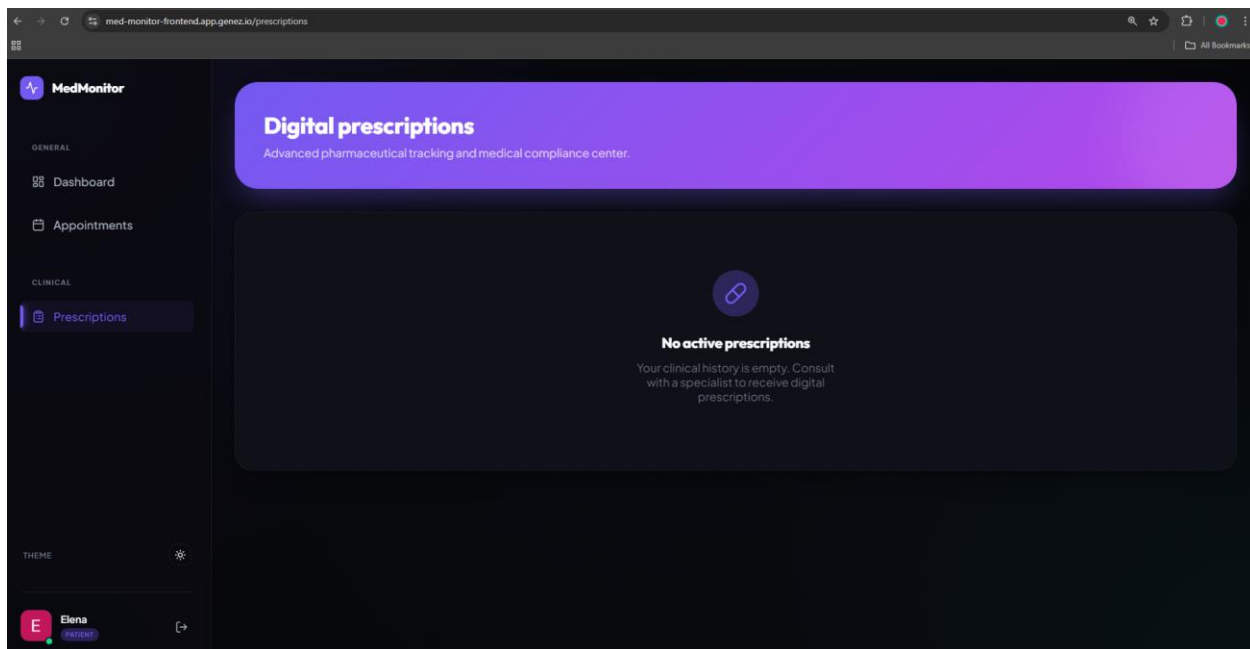


Fig. 6 Vizualizarea prescripțiilor digitale din perspectiva pacientului

Endpoint-uri API și Gestiunea Securității

Accesul la datele sensibile despre tratamente este protejat prin reguli stricte de autorizare, asigurând confidențialitatea actului medical:

- **GET /prescriptions:** Acest endpoint este cel mai frecvent utilizat de către pacient. Backend-ul verifică identitatea extrasă din JWT și returnează **exclusiv** prescripțiile aparținând acelui utilizator. Un pacient nu are posibilitatea tehnică de a interoga sau vizualiza tratamentele altor persoane, sistemul returnând o listă goală sau o eroare în cazul oricărei tentative de acces neautorizat.
- **POST /prescriptions:** Funcționalitate restricționată exclusiv rolului de **Doctor**. Aceasta permite crearea înregistrării în baza de date și legarea ei de ID-ul pacientului.
- **GET /prescriptions/:id:** Permite vizualizarea detaliată a unei singure rețete. Accesul este permis medicului care a emis-o și pacientului care a primit-o, asigurând un canal de comunicare securizat și privat.

Prin acest modul, **Med-Monitor** elimină riscul de pierdere a documentelor medicale și oferă pacientului un control total asupra propriului istoric terapeutic, într-un format modern și ușor de utilizat.

2.6 Frontend: Design Glassmorphism și Structura React

Interfața utilizator a platformei **Med-Monitor** a fost proiectată pentru a oferi o experiență estetică de ultimă generație, punând accent pe claritate, profunzime și o ierarhie vizuală modernă. Alegerea tehnologică s-a concentrat pe performanță și control total asupra designului, evitând constrângerile librăriilor de componente predefinite.

Estetica Vizuală: Glassmorphism și Dualitatea de Temă (Dark/Light)

Spre deosebire de aplicațiile medicale tradiționale care adoptă un stil utilitar rigid, **Med-Monitor** utilizează conceptul de **Glassmorphism**. Acest stil vizual emulează aspectul sticlei mate, oferind interfeței o notă „premium” și aerisită.

- **Implementarea CSS Pure:** Interfața este construită integral cu **CSS pur**, utilizând funcția backdrop-filter: blur(10px) și fundaluri semi-transparente (rgba). Această abordare a permis controlul total asupra reflexiilor și texturilor, fără a depinde de framework-uri externe.
- **Sistemul de Teme (Dark & Light Mode):** Aplicația dispune de un mecanism de tematică duală, adaptabil preferințelor utilizatorului.
 - În **Light Mode**, elementele de tip "glass" capătă o strălucire cristalină, optimizând lizibilitatea în medii puternic luminate.
 - În **Dark Mode**, interfața folosește tonuri profunde de gri și albastru închis, reducând oboseala oculară și evidențiind accentele de neon ale datelor medicale.
 - Trecerea între cele două moduri se face instantaneu, prin manipularea **variabilelor CSS (Custom Properties)** la nivelul rădăcinii documentului, asigurând o tranziție vizuală fluidă pe toate componentele.

Arhitectura Aplicației React

Structura software a frontend-ului respectă bunele practici de dezvoltare modulară, fiind organizată pe două paliere principale: **Pagini** (containere de logică) și **Componente Reutilizabile** (atomi de UI precum butoane, carduri, input-uri).

Utilizând React Router, aplicația implementează un sistem de **Protected Routes** (Rute Protejate). Acesta funcționează ca un gardian la nivel de interfață:

- Înainte de a randa o pagină (ex: Dashboard-ul de Medic), sistemul verifică starea de autentificare și rolul stocat în contextul aplicației.
- Dacă un utilizator cu rol de **Patient** încearcă să acceseze manual o rută de administrare, sistemul îl redirecționează automat către pagina de start, prevenind expunerea unor elemente de interfață neautorizate.

Comunicarea cu API-ul și Interceptorii Axios

Pentru interacțiunea cu backend-ul Go, s-a utilizat librăria **Axios**, configurată strategic pentru a automatiza securitatea comunicației:

1. **Injectarea Automată a JWT:** Prin utilizarea interceptorilor de cerere (*request interceptors*), header-ul Authorization: Bearer <token> este adăugat automat fiecărui apel API.
2. **Tratarea Erorilor (401 Unauthorized):** Interceptorii de răspuns monitorizează constant statusul venit de la server. În cazul în care un token expiră (Eroare 401), aplicația reacționează imediat, curățând starea locală și redirecționând utilizatorul către pagina de Login pentru o nouă autentificare prin Google.

Această arhitectură frontend asigură un echilibru perfect între o estetică vizuală sofisticată și o funcționare tehnică robustă, oferind utilizatorului un mediu digital sigur, personalizabil și plăcut.

2.7 Disponibilitatea Sistemului și Accesibilitate Live

Pentru a demonstra portabilitatea și robustețea soluției, aplicația **Med-Monitor** a fost configurată pentru a fi ușor de deploy-at și testat. Aceasta este disponibilă atât ca mediu de producție live, cât și ca mediu de dezvoltare local prin tehnologii de containerizare.

Aplicația este deployed și poate fi accesată în timp real la următoarea adresă:

<https://med-monitor-frontend.app.genez.io/>

Această versiune online este complet funcțională, permițând testarea fluxului de autentificare Google OAuth 2.0 și explorarea designului Glassmorphism în ambele moduri de culoare (Dark/Light).

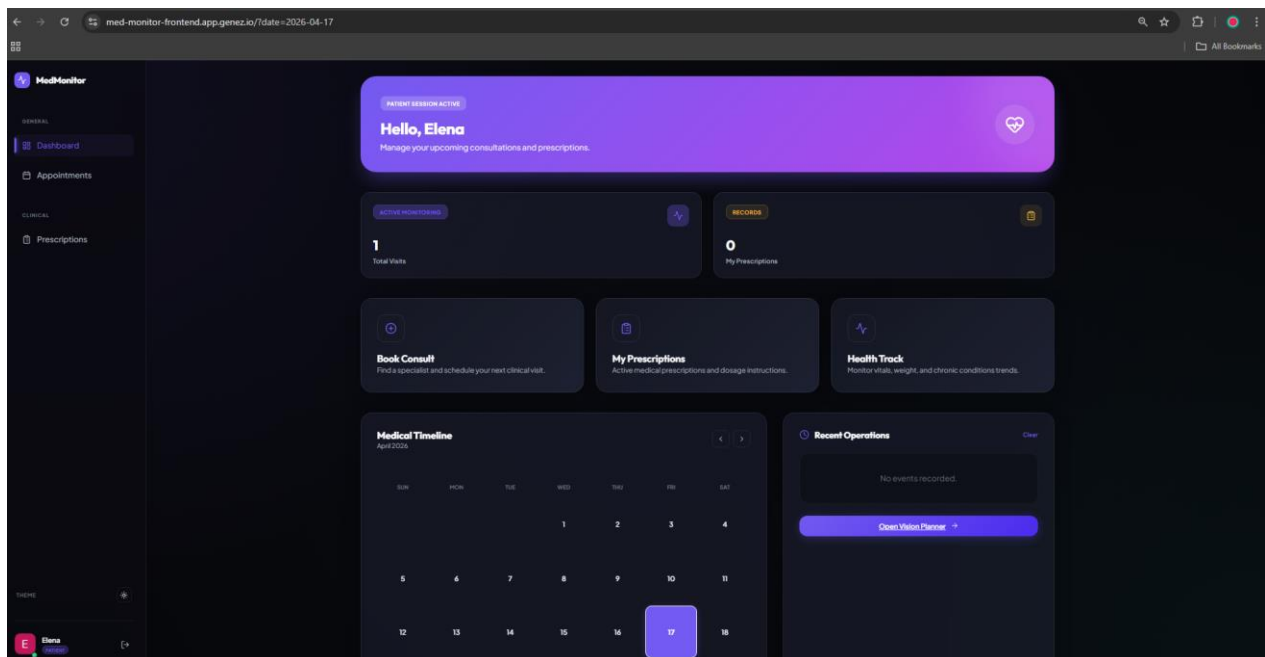


Fig. 7 Dashboard-ul aplicației

2.8 Securitate si Bune Practici

Sistemul **Med-Monitor** a fost proiectat având securitatea ca prioritate centrală, adoptând principiul *Security by Design*. Având în vedere natura sensibilă a datelor medicale vehiculate, au fost implementate multiple straturi de protecție pentru a asigura confidențialitatea, integritatea și disponibilitatea informațiilor.

Straturi de Protecție și Controlul Accesului

Pentru a minimiza suprafața de atac și a preveni accesul neautorizat, au fost integrate următoarele mecanisme tehnice:

- **Arhitectură JWT Stateless:** Autentificarea nu depinde de sesiuni stocate pe server, ceea ce elimină riscul de *session hijacking* la nivel de memorie server și facilitează scalarea orizontală. Fiecare cerere este self-contained și verificată matematic prin semnătura digitală a jetonului.
- **Validarea Riguroasă a Google ID Token:** Previne falsificarea identității prin verificarea în backend a jetoanelor furnizate de Google, asigurându-se că utilizatorul este cine pretinde a fi înainte de a-i emite un acces local.
- **Casbin RBAC (Role-Based Access Control):** Securitatea este aplicată uniform la nivel de middleware. Fiecare endpoint este protejat automat; în cazul unei tentative de acces neautorizat, sistemul blochează cererea și returnează eroarea 403 Forbidden înainte ca aceasta să consume resurse de calcul sau să interogheze baza de date.
- **CORS (Cross-Origin Resource Sharing) Configurat:** Backend-ul este configurat să accepte cereri exclusiv de la originea controlată a frontend-ului, prevenind atacurile de tip *Cross-Site Request Forgery* (CSRF) din alte domenii malițioase.
-

Standarde de Implementare și Gestiunea Datelor

Pe lângă securitatea activă, proiectul respectă bunele practici de codare și gestionare a infrastructurii:

- **Standardizarea snake_case JSON:** Utilizarea unui format standardizat pentru comunicarea API previne expunerea accidentală a denumirilor câmpurilor interne din baza de date sau din structurile Go, oferind un strat suplimentar de abstractizare.
- **Izolarea Credențialelor (Environment Variables):** Toate datele sensibile (chei secrete, URL-uri de baze de date, Client ID-uri) sunt gestionate prin fișiere .env și variabile de mediu. Acestea nu sunt incluse în repository-ul public (GitHub), eliminând riscul de scurgere a cheilor de securitate.
- **Prevenirea SQL Injection:** Prin utilizarea GORM ca strat de abstractizare (ORM), toate interogările către PostgreSQL sunt parametrizate automat, neutralizând una dintre cele mai comune vulnerabilități la nivelul bazelor de date.

Acest set de bune practici garantează că **Med-Monitor** nu este doar o platformă funcțională, ci și un ecosistem digital robust, aliniat la standardele moderne de securitate cibernetică.

3. Concluzie

Med-Monitor demonstrează cum un proiect full-stack modern poate fi realizat cu o stivă tehnologică coerentă și performantă. Alegerea limbajului Go împreună cu framework-ul Gin pentru backend oferă o performanță remarcabilă la un cost de resurse redus, aspect esențial pentru sistemele medicale care trebuie să gestioneze sute de cereri simultane în siguranță. În același timp, utilizarea React cu TypeScript garantează o interfață robustă, fiind o bază solidă, ușor de menținut și extins pe termen lung.

Implementarea Casbin RBAC reprezintă un punct forte al arhitecturii, deoarece politicile de acces sunt externalizate din codul sursă. Această abordare permite unui administrator să modifice permisiunile fără a necesita o nouă versiune a aplicației sau un nou proces de deploy. Totodată, autentificarea prin Google OAuth 2.0 elimină responsabilitatea gestionării parolilor de către echipa de dezvoltare, transferând securitatea procesului către o platformă de încredere și oferind utilizatorului o experiență de acces simplificată.

Din punct de vedere al experienței utilizatorului, designul Glassmorphism implementat exclusiv în CSS pur reprezintă o abordare distinctivă. Prin evitarea dependențelor externe, s-a reușit reducerea dimensiunii pachetelor de date și oferirea unui aspect vizual modern, care se diferențiază clar de aplicațiile construite cu framework-uri UI generice. Mai mult, containerizarea cu Docker și deploy-ul pe Genezio Cloud asigură portabilitatea și reproductibilitatea mediului, calități indispensabile într-un ecosistem medical unde fiabilitatea sistemului este critică.

Pentru viitor, proiectul poate evolua prin integrarea unor sisteme de notificări push (email sau SMS) pentru confirmarea programărilor, adoptarea standardelor HL7 FHIR pentru interoperabilitate cu laboratoare externe sau implementarea unui modul de telemedicină cu videoconferință integrată. De asemenea, dezvoltarea unei aplicații mobile native în React Native și a unui dashboard de analiză cu grafice de tendințe ar putea extinde utilitatea platformei pentru toți actorii implicați.

În concluzie, Med-Monitor este un sistem matur conceptual, cu o arhitectură clară, securitate multi-strat și o experiență de utilizare modernă, fiind pregătit să servească drept fundament solid pentru o soluție medicală de producție.

Bibliografie

- [1] „Documentation,” [Interaktiv]. Available: <https://go.dev/doc/>.
- [2] „Effective Go,” [Interaktiv]. Available: https://go.dev/doc/effective_go#goroutines.
- [3] „Introduction in GIN,” [Interaktiv]. Available: <https://gin-gonic.com/docs/introduction/>.
- [4] „Model Binding and Validation,” [Interaktiv]. Available: <https://gin-gonic.com/docs/examples/model-binding-and-validation/>.
- [5] „Features and Design,” [Interaktiv]. Available: <https://github.com/julienschmidt/httprouter#features-and-design>.
- [6] „Thinking in React,” [Interaktiv]. Available: <https://react.dev/learn/thinking-in-react>.
- [7] „Render and Commit,” [Interaktiv]. Available: <https://react.dev/learn/render-and-commit>.
- [8] „The TypeScript Handbook,” [Interaktiv]. Available: <https://www.typescriptlang.org/docs/handbook/intro.html>.
- [9] „Announcing TypeScript 5.0,” [Interaktiv]. Available: <https://devblogs.microsoft.com/typescript/announcing-typescript-5-0/>.
- [10] „Features,” [Interaktiv]. Available: <https://vitejs.dev/guide/features.html>.
- [11] „PostgreSQL: The World's Most Advanced Open Source Relational Database,” [Interaktiv]. Available: <https://www.postgresql.org/docs/>.
- [12] „GORM Guides,” [Interaktiv]. Available: <https://gorm.io/docs/index.html>.
- [13] „Migration,” [Interaktiv]. Available: <https://gorm.io/docs/migration.html>.
- [14] „Transactions,” [Interaktiv]. Available: <https://gorm.io/docs/transactions.html>.
- [15] „Overview,” [Interaktiv]. Available: <https://casbin.org/docs/overview>.
- [16] „How it works,” [Interaktiv]. Available: <https://casbin.org/docs/how-it-works>.
- [17] „RBAC,” [Interaktiv]. Available: <https://casbin.org/docs/rbac>.
- [18] „Role Definition,” [Interaktiv]. Available: <https://casbin.org/docs/role-definition>.
- [19] „Adapters,” [Interaktiv]. Available: <https://casbin.org/docs/adapters>.
- [20] „OAuth 2.0,” [Interaktiv]. Available: <https://oauth.net/2/>.
- [21] „Protocols,” [Interaktiv]. Available: <https://developers.google.com/identity/protocols/oauth2>.
- [22] „JSON Web Token (JWT),” [Interaktiv]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [23] „Introduction,” [Interaktiv]. Available: <https://jwt.io/introduction/>.
- [24] „JSON WEB Tokens,” [Interaktiv]. Available: <https://auth0.com/docs/secure/tokens/json-web-tokens>.
- [25] „Docker Overview,” [Interaktiv]. Available: <https://docs.docker.com/get-started/overview/>.
- [26] „Use containers to Build, Share and Run your applications,” [Interaktiv]. Available: <https://www.docker.com/resources/what-container/>.
- [27] „Docker Compose,” [Interaktiv]. Available: <https://docs.docker.com/compose/>.
- [28] „Volumes,” [Interaktiv]. Available: <https://docs.docker.com/storage/volumes/>.
- [29] „Building best practices,” [Interaktiv]. Available: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/.
- [30] „Genezio Overview,” [Interaktiv]. Available: <https://genezio.com/docs/genezio-overview/>.
- [31] „Genezio Cloud,” [Interaktiv]. Available: <https://genezio.com/>.
- [32] „The Genezio Blog,” [Interaktiv]. Available: <https://genezio.com/blog/>.
- [33] „UX Design Institute Blog,” [Interaktiv]. Available: <https://www.uxdesigninstitute.com/blog/>.

- [34] „backdrop-filter,” [Interactiv]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/backdrop-filter>.
- [35] „backdrop-filter,” [Interactiv]. Available: <https://css-tricks.com/almanac/properties/b/backdrop-filter/>.
- [36] „Articles,” [Interactiv]. Available: <https://web.dev/articles/>.
- [37] „Resources for Developers,” [Interactiv]. Available: <https://developer.mozilla.org/en-US/>.